캡스톤 공방전 케이스 S1~S10
진행·검증 가이드

한빛정밀 4계층 SOAR 캡스톤 · 블루팀(방어) 관점 · 작성일 2026-10-16

이 문서는 사진 속 "케이스 10종(S1~S10, 문서 153~162)"이 **어떻게 진행되고**, **무엇을 결과물로 확인해야 하며**, 전체가 **어떤 방식의 프로젝트인지**를 케이스별 챕터로 정리한 것이다.

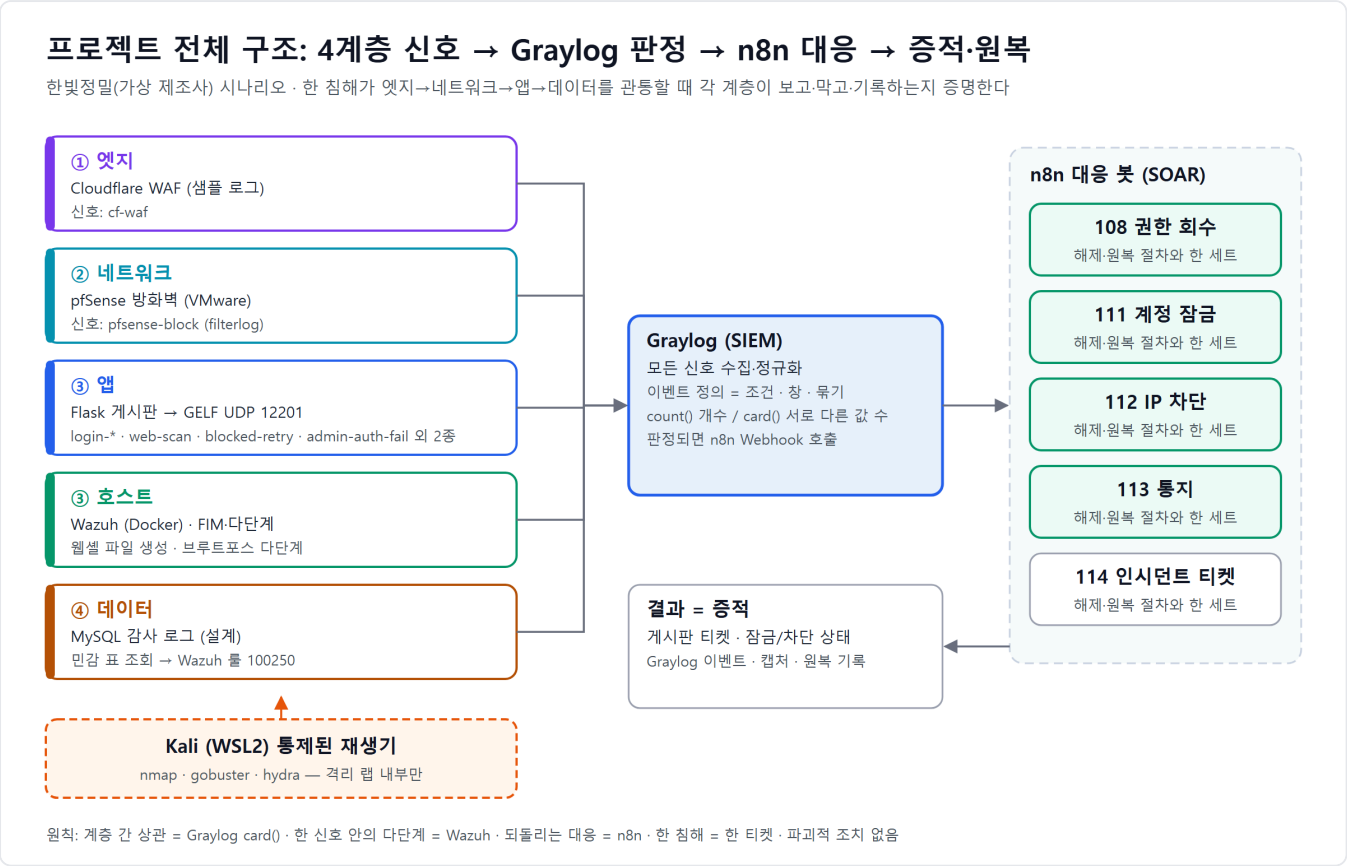
목차

1. 프로젝트는 어떤 방식으로 돌아가는가
2. 읽는 법: 신호 · 조건 · 창 · 묶기 · 대응 봇
3. 케이스 하나를 검증하는 공통 순서
4. S1 저속 브루트포스
5. S2 심야 로그인 성공
6. S3 한 계정 다중 출발지
7. S4 뚫린 계정(실패 뒤 성공)
8. S5 차단 우회 시도
9. S6 관리자 API 폭주
10. S7 웹셸 업로드 후 접근
11. S8 스캐너→브루트포스 연쇄
12. S9 권한 상승 후 조회
13. S10 전 계층 관통
14. 통합 스토리라인과 최종 결과물
15. 팀이 먼저 정해야 할 것 · 출처

> **근거와 한계** — 케이스 정의(조건·창·묶기·대응)는 노션 [168] 정리판의 "케이스 10종" 표와 저장소 `aleph-soar-capstone/docs/03-scenarios.md` 를 따랐다. 신호가 실제로 만들어지는 위치는 게시판 코드(`_7_board_test`)에서 확인했다. 케이스별 원본 문서 [153]~[162] 본문은 직접 열람하지 못했으므로, 세부 절차는 그 문서와 대조해 보완한다. 원본의 "실측" 수치는 원본 환경의 값이며 우리 팀의 검증 결과가 아니다.

1. 프로젝트는 어떤 방식으로 돌아가는가

한 침해가 엣지 → 네트워크 → 앱 → 데이터를 관통할 때, 각 계층이 그것을 보고·막고·기록하는지를 증거로 증명하는 프로젝트다.



네 가지 원칙

- **계층별 방어**: 한 계층이 놓친 것을 다음 계층이 잡는 심층 방어.
- **신호를 하나로 잇기**: 출처가 다른 신호를 Graylog에서 상관(card)해 "한 침해 = 한 인시던트"로 묶는다. 한 신호 안의 다단계 판정은 Wazuh가 맡는다.
- **되돌릴 수 있는 대응**: n8n의 잠금·차단·권한 회수는 모두 해제 절차와 한 세트다. 파괴적 조치는 하지 않는다.
- **증적으로 평가**: "실제로 동작했는가"를 로그·이벤트·티켓·원복 기록으로 채점한다.

구성 요소

계층	도구	이 프로젝트에서 하는 일
① 엣지	Cloudflare WAF (샘플)	외부 노출 없이 WAF 로그 샘플을 SIEM에 넣어 탐지 연습 (`cf-waf`)
② 네트워크	pfSense	공장↔본사 경계, 차단 기록 `filterlog` → Graylog (`pfsense-block`)
③ 앱	Flask 게시판	로그인·관리 API·등급 이벤트를 GELF 신호로 신고
③ 호스트	Wazuh	파일 생성·변경(FIM)과 호스트 로그 다단계 탐지
④ 데이터	MySQL	최소권한 계정 + 감사 로그로 민감 표 조회 탐지 (설계 단계)
상관	Graylog	이벤트 정의가 조건·창·묶기를 판정해 대응을 부른다
대응	n8n	108 권한 회수 · 111 계정 잠금 · 112 IP 차단 · 113 통지 · 114 인시던트 티켓

검증	Kali (WSL2)	격리 랩 안에서만 통제된 방식으로 신호를 재생해 방어를 검증

안전 원칙(절대) — 격리 랩 전용, 운영 서버 미접촉 · 문서용 대역 IP와 `zz_` 테스트 계정만 사용 · 웹셸 검증은 실행되지 않는 무해한 파일로 · 비밀번호·개인정보·시도된 키 값은 저장하지 않는다.

2. 읽는 법: 신호 · 조건 · 창 · 묶기 · 대응 봇

카드 한 장은 "어떤 신호가, 어떤 키로 묶여, 정해진 시간 안에, 조건을 넘으면, 어떤 봇이 대응한다"를 뜻한다.

케이스 10종 한눈에 보기 (S1~S10 · 원본 문서 153~162)

Graylog 이벤트 정의 = 신호 + 조건 + 시간창 + 묶기 → 조건 충족 시 n8n 대응 봇 호출

케이스	이름	계층	신호	조건	창	묶기	대응
S1	저속 브루트포스	앱	login-bruteforce	count()>8	30분	username	111 잠금
S2	심야 로그인 성공	앱	night_login	count()>0	5분	username	114 티켓
S3	한 계정 다중 출발지	앱	login-success	card(src_ip)>1	5분	username	114 티켓
S4	뚫린 계정(실패 뒤 성공)	앱	login-*	card(outcome)>1 · count()>5	10분	username	111+114
S5	차단 우회 시도	앱	blocked-retry	count()>19	10분	src_ip	114 티켓
S6	관리자 API 폭주	앱	admin-auth-fail	count()>9	5분	src_ip	112 차단
S7	웹셸 업로드 후 접근	호스트 앱	signal_kind	card(signal_kind)>1	10분	(없음)	114 티켓
S8	스캐너→브루트포스 연쇄	앱	web-scan + login	card(rule)>1	15분	src_ip	112+114
S9	권한 상승 후 조회	앱 데이터	priv + gold-access	card(rule)>1	30분	username	108+114
S10	전 계층 관통	네트워크 앱	pfsense+web+login	card(rule)>2	15분	src_ip	112+114+pf

대응 봇: 108 권한 회수 · 111 계정 잠금 · 112 IP 차단 · 113 통지 · 114 인시던트 티켓 · pf = pfSense 경계 차단
S1~S6은 신호 하나의 개수(count) · S7~S10은 서로 다른 신호의 조합(card) — 뒤로 갈수록 여러 계층을 이어야 한다

용어	뜻	예
신호	Graylog에 도착하는 이벤트의 `rule` 값	`login-bruteforce`
조건 `count()`	창 안에서 해당 신호가 몇 건인지	`count()>8` = 9건 이상
조건 `card(x)`	창 안에서 필드 x의 **서로 다른 값**이 몇 개인지	`card(src_ip)>1` = 출발지 2곳 이상
창	세는 시간 범위	30분
묶기	무엇 기준으로 따로 세는지	username별, src_ip별
대응 봇	조건 충족 시 n8n이 실행하는 조치	111 잠금 + 114 티켓

경계값 주의: 조건은 모두 "초과"(`>`)다. `count()>8`은 8건에서는 발동하지 않고 9건째에 발동한다. 검증할 때는 반드시 경계 바로 아래와 바로 위를 모두 확인한다.

3. 케이스 하나를 검증하는 공통 순서

모든 케이스는 같은 9단계로 진행하고, 같은 기록 틀로 남긴다. 그래야 누가 맡아도 결과를 비교할 수 있다.



세 번 돌리기

입력	기대 결과
정상 입력	Graylog 이벤트 0건, 대응 0건 (오탐 없음)
임계 바로 아래	신호는 도착하지만 이벤트 0건
임계 충족	이벤트 1건 → n8n 실행 성공 → 대상 상태 변화 → 티켓 1건

추가로 확인할 것: 다른 사용자·IP가 섞여도 묶기 키대로 따로 세는가, 같은 입력을 다시 보내도 티켓·잠금이 중복되지 않는가.

검증 기록 틀 (케이스마다 1장)

시나리오 ID · 담당 · 일시 · 커밋	
대상 랩과 사전 상태	
입력 이벤트 · 상관 키 · 시간 순서	
정상 · 임계 미만 결과	
임계 충족 결과	로그 / 탐지 이벤트 / 대응 상태 / 티켓
중복 · 혼합 입력 결과	
원복 절차와 원복 후 상태	

증적 경로 · 판정 · 남은 문제	

원복 순서 (모든 케이스 공통)

IP 차단 해제 → 계정 잠금 해제 → 권한 원복 · 티켓 종료 → 샘플 로그 · 감사 로그 · 규칙 원복. 원복 후 사전 상태와 같은지 다시 확인한다.

S1 · 저속 브루트포스

앱 | 원본 문서 153 | 대응 111 계정 잠금 | MVP 우선 후보

짧은 시간 임계를 피해 천천히, 오래 한 계정을 두드리는 비밀번호 추측을 계정 기준으로 넓게 세어 잡는다.

무엇을 막나

2분 단위 같은 짧은 창의 임계를 피하려고 간격을 두고 시도하는 공격(ATT&CK T1110.001). 짧은 창 규칙만 있으면 놓친다.

판정 조건

신호	`login-bruteforce` — 로그인 실패, 잠긴 계정에 대한 시도
조건 · 창 · 묶기	`count()>8` · 30분 · username
대응	111 계정 잠금
신호 출처	`controllers/auth_controller.py` 로그인 실패 시 GELF 전송(username·src_ip·count), 잠긴 계정 시도 시 `locked=1`

진행 방식

1. 테스트 계정 `zz_`을 준비하고 잠금 상태가 아님을 기록한다.
2. 30분 창 안에서 같은 계정의 로그인 실패를 8건(임계 미만) → 이벤트가 없는지 확인한다.
3. 9건째를 넣어 이벤트가 생기고 n8n 111이 계정을 잠그는지 확인한다.
4. 잠긴 뒤 시도는 `locked=1`로 계속 기록되는지 확인한다.

확인할 결과물

- Graylog 검색 `rule:login-bruteforce username:zz_…` 결과 9건
- Graylog Alerts의 S1 이벤트 1건, n8n 실행 기록 성공
- 게시판 관리 화면에서 계정 상태 "잠김", 로그인 응답 423
- 원복: 잠금 해제 후 정상 로그인 성공

흔한 실패 — 다른 계정 실패가 섞이면 username별로 따로 세어야 한다. 짧은 창 규칙(2분 6회)과 함께 쓰면 같은 공격에 두 번 대응하지 않는지 중복도 확인한다.

S2 · 심야 로그인 성공

앱 | 원본 문서 154 | 대응 114 티켓

실패가 아니라 "성공"이라도, 업무 시간이 아닌 심야에 일어났다면 사람이 확인하도록 티켓을 만든다.

무엇을 막나

탈취한 정상 계정으로 감시가 약한 시간에 들어오는 접근(T1078 유효 계정).

판정 조건

신호	`night_login` — 심야 시간대의 로그인 성공
조건 · 창 · 묶기	`count()>0` · 5분 · username (1건만 있어도 발동)
대응	114 인시던트 티켓 (자동 차단 없음)
신호 출처	게시판은 `login-success`를 보낸다. `night_login`은 성공 로그 중 심야 시간대만 골라 만든 파생 신호로, 만드는 위치(Graylog 파이프라인 또는 Wazuh 규칙)를 팀이 정해야 한다.

진행 방식

1. "심야"의 기준(예: 22시~06시 KST)을 정하고 문서에 적는다.
2. 낮 시간 로그인 성공 → 티켓이 생기지 않는지 확인한다.
3. 심야 시간대 로그인 성공 1건 → 티켓 1건이 생기는지 확인한다.

확인할 결과물

- `night_login` 신호 1건(시각·username·src_ip 포함)
- S2 이벤트 1건과 티켓 1건
- 낮 시간 성공 로그는 신호 0건

흔한 실패 — 시간 판정 기준이 UTC인지 KST인지 어긋나 9시간 밀린다(실제로 매니저 규칙 주석에 같은 혼란이 있다). Wazuh의 시간 조건은 로그 시각이 아니라 매니저의 현재 시각으로 판정한다. 검증을 위해 시간대를 임시로 바꿨다면 반드시 원복한다.

S3 · 한 계정 다중 출발지

앱 원본 문서 155 대응 114 티켓

한 계정이 짧은 시간에 서로 다른 출발지 IP에서 로그인에 성공하면, 계정이 공유되거나 탈취됐다는 신호로 본다.

판정 조건

신호	`login-success`
조건 · 창 · 묶기	`card(src_ip)>1` · 5분 · username
대응	114 티켓
신호 출처	`auth_controller.py` 로그인 성공 시 GELF(username·src_ip·role)

진행 방식

1. 같은 계정, 같은 IP에서 여러 번 성공 → 이벤트 없음.
2. 같은 계정이 5분 안에 두 번째 IP에서 성공 → 이벤트·티켓 1건.
3. 서로 다른 계정이 각각 다른 IP에서 성공 → 이벤트 없음(묶기 확인).

확인할 결과물

- Graylog 집계에서 해당 username의 서로 다른 src_ip 2개
- S3 이벤트 1건, 티켓 1건(두 IP가 티켓에 함께 기록)

먼저 정할 것 — 랩에서는 모든 요청이 127.0.0.1이나 같은 게이트웨이로 보일 수 있다. 앱이 출발지 IP를 어떻게 읽는지(프록시 헤더 신뢰 여부)를 먼저 정하고, 테스트용 출발지 구분 방법을 문서에 적는다. 정상 사용자가 PC와 휴대폰을 함께 쓰는 경우의 오탐 기준도 정한다.

S4 · 뚫린 계정(실패 뒤 성공)

앱 | 원본 문서 156 | 대응 111 잠금 + 114 티켓 | MVP 우선 후보

여러 번 실패한 뒤 결국 성공했다면 비밀번호가 뚫렸을 가능성이 높으므로, 계정을 잠그고 티켓을 연다.

판정 조건

신호	`login-*` (실패 `login-bruteforce` + 성공 `login-success`)
조건 · 창 · 묶기	`card(outcome)>1` 그리고 `count()>5` · 10분 · username
대응	111 계정 잠금 + 114 티켓

진행 방식

1. 실패만 6건 이상 → 결과 종류가 1개뿐이라 S4 이벤트 없음.
2. 실패 5건 + 성공 1건(합계 6건) → 이벤트, 잠금, 티켓.
3. 성공 후 계정이 잠겨 다음 로그인이 거부되는지 확인.

확인할 결과물

- 같은 username의 실패·성공 신호와 시각 순서
- S4 이벤트 1건, 계정 잠김, 티켓 1건
- 원복: 잠금 해제, 테스트 계정 비밀번호 재설정

코드에서 확인한 문제 — 조건이 `outcome` 필드의 서로 다른 값을 세는데, 현재 게시판의 로그인 GELF에는 `outcome` 필드가 없다(실패·성공은 `rule` 값으로만 구분된다). 필드를 추가하거나 조건을 `card(rule)` 로 바꿔야 한다. 또한 서로 다른 값의 개수만으로는 "실패 **뒤에** 성공"이라는 순서를 증명할 수 없으므로 순서 확인 방법을 따로 정한다. 원본 [156]에서도 Wazuh 합성 규칙이 발동하지 않은 실측 기록이 있다.

S5 · 차단 우회 시도

앱 | 원본 문서 157 | 대응 114 티켓

이미 차단된 IP가 계속 두드린다면 공격이 멈추지 않은 것이다. 403만 주고 끝내지 않고 지속성을 기록해 사람이 판단하게 한다.

판정 조건

신호	`blocked-retry` — 차단 목록에 있는 IP의 요청
조건 · 창 · 묶기	`count()>19` · 10분 · src_ip
대응	114 티켓 (경계 차단 등 상위 조치 검토)
신호 출처	`app.py` 요청 전 차단 확인 단계. 관리자 API와 유효한 자동화 키 요청은 예외로 둔다(자기 차단 방지).

진행 방식

1. 테스트 IP를 차단 목록에 넣고 사전 상태를 기록한다.
2. 그 IP의 요청 19건 → 모두 403, 이벤트 없음.
3. 20건째 → 이벤트·티켓 1건.
4. 관리자 API 요청은 차단되지 않고 신호도 생기지 않는지 확인한다.

확인할 결과물

- `rule:blocked-retry` 20건, 응답 코드 403
- S5 이벤트·티켓 1건
- 원복: 차단 해제 후 정상 응답

흔한 실패 — n8n이 127.0.0.1을 차단해 같은 호스트의 자동화 호출까지 막히는 자기 차단. 코드의 예외 규칙이 실제로 동작하는지 같이 확인한다.

S6 · 관리자 API 폭주

앱 | 원본 문서 158 | 대응 112 IP 차단

관리자·보안 API의 키 인증 실패가 짧은 시간에 몰리면 키를 추측하는 공격으로 보고 출발지 IP를 차단한다.

판정 조건

신호	`admin-auth-fail` — API 키 인증 실패(401)
조건 · 창 · 묶기	`count()>9` · 5분 · src_ip
대응	112 IP 차단
신호 출처	`security_controller.py` 키 검증 실패 시 GELF와 Wazuh용 로그를 함께 남긴다. **시도된 키 값은 절대 남기지 않는다.**

진행 방식

1. 잘못된 키로 9건 → 이벤트 없음.
2. 10건째 → 이벤트, 해당 IP 차단.
3. 차단 후 같은 IP의 일반 요청은 403(S5 신호로 이어짐).

확인할 결과물

- `rule:admin-auth-fail` 10건, 로그에 키 값이 없는지 확인
- S6 이벤트, 차단 목록에 IP 추가
- 원복: 차단 해제

흔한 실패 — n8n 자신의 호출이 키 오설정으로 실패하면 n8n 호스트가 스스로 차단될 수 있다. 자동화 IP 예외와 키 설정을 먼저 점검한다.

S7 · 웹셸 업로드 후 접근

호스트 앱 원본 문서 159 대응 114 티켓

웹 디렉터리에 실행형 파일이 생긴 직후 그 경로로 웹 접근이 일어나면, 웹셸이 심어지고 사용된 흐름으로 본다.

판정 조건

신호	`signal_kind` — 파일 생성(Wazuh FIM)과 웹 접근, 두 종류
조건 · 창 · 묶기	`card(signal_kind)>1` · 10분 · 원본에는 묶기 키 없음
대응	114 티켓
신호 출처	호스트: Wazuh FIM이 웹 디렉터리의 실행형 확장자 파일 생성을 탐지(게시판 규칙 100220~100222 계열). 앱: 해당 경로 접근 기록. 두 신호에 `signal_kind` 값을 붙여 Graylog로 보내는 경로는 팀이 만들어야 한다.

진행 방식

1. 실행되지 않는 무해한 테스트 파일을 감시 폴더에 만든다 → FIM 경보 1건, S7 이벤트 없음.
2. 10분 안에 그 경로로 웹 요청 → 신호 종류 2개, S7 이벤트·티켓.
3. 테스트 파일 삭제와 삭제 경보까지 기록한다.

확인할 결과물

- Wazuh 대시보드 FIM 경보(파일 경로·시각)
- Graylog에서 같은 경로의 두 신호
- S7 이벤트·티켓, 원복 후 파일 없음

먼저 정할 것 — 묶기 키가 없으면 서로 무관한 파일 생성과 웹 접근이 합쳐져 오탐이 난다. 파일 경로나 자산 ID로 묶는 기준을 정한다.

S8 · 스캐너 → 브루트포스 연쇄

앱 | 원본 문서 160 | 대응 112 차단 + 114 티켓

같은 출발지가 먼저 웹을 탐색하고 이어서 로그인을 반복하면, 정찰 뒤 공격으로 이어진 하나의 흐름으로 본다.

판정 조건

신호	`web-scan` (404 대량 발생) + 인증 신호(`login-bruteforce`)
조건 · 창 · 묶기	`card(rule)>1` · 15분 · src_ip
대응	112 IP 차단 + 114 티켓
신호 출처	`app.py` 404 응답 시 `web-scan` (관리자 API 제외) · `auth_controller.py` 로그인 실패

진행 방식

1. 한 출발지에서 404만 발생 → 신호 종류 1개, 이벤트 없음.
2. 같은 출발지에서 15분 안에 로그인 실패 추가 → 이벤트, 차단, 티켓.
3. 404와 로그인 실패가 서로 다른 출발지에서 나면 → 이벤트 없음.

확인할 결과물

- 같은 src_ip의 두 rule 값
- S8 이벤트 1건, 차단, 티켓 1건
- S1과 동시에 발동할 때 티켓이 하나로 묶이는지

흔한 실패 — 서로 다른 rule의 개수만으로는 "탐색 **뒤에** 로그인"이라는 순서를 보장하지 못한다. 순서가 반대여도 발동하므로 순서 확인 기준을 정한다.

S9 · 권한 상승 후 조회

앱 데이터 원본 문서 161 대응 108 권한 회수 + 114 티켓

허가되지 않은 권한 상승이 일어난 계정이 곧이어 상위 등급 영역을 실제로 열람했다면, 권한을 회수하고 티켓을 연다.

판정 조건

신호	`priv-unauthorized-admin` (승인 없는 관리자 등급) + `gold-access` (골드 영역 열람)
조건 · 창 · 묶기	`card(rule)>1` · 30분 · username
대응	108 권한 회수 + 114 티켓
신호 출처	`admin_controller.py` ·권한 회수 봇이 비인가 관리자 신호를, `gold_controller.py` 가 열람 신호(계정명·등급·출발지만, 본문 미기록)를 보낸다. 데이터 계층 짝은 MySQL 감사 → Wazuh 규칙 100250(설계).

진행 방식

1. 테스트 계정 등급을 승인 없이 올린 상태를 만든다 → 신호 1종, 이벤트 없음.
2. 그 계정이 30분 안에 골드 영역을 열람 → 이벤트, 권한 회수, 티켓.
3. 정상 승인된 골드 계정의 열람은 이벤트가 없어야 한다.

확인할 결과물

- 같은 username의 두 신호
- 권한 회수 후 해당 계정의 골드 영역 요청 403
- 티켓 1건, 원복 시 등급을 사전 상태로

확장 — 데이터 계층(MySQL 감사)까지 이으려면 감사 로그 수집 경로와 비밀값 제외 방법을 먼저 정해야 한다. 현재는 설계 단계다.

S10 · 전 계층 관통

네트워크 앱 원본 문서 162 대응 112 + 114 + pfSense 경계 차단

같은 출발지의 신호가 경계(pfSense)·웹·인증 등 세 종류 이상에서 동시에 나타나면 계층을 관통하는 침해로 보고 앱과 경계에서 함께 막는다.

판정 조건

신호	`pfsense-block` (filterlog) + `web-scan` + 로그인 신호
조건 · 창 · 묶기	`card(rule)>2` · 15분 · src_ip
대응	112 앱 IP 차단 + 114 티켓 + pfSense 경계 차단
신호 출처	pfSense filterlog → Graylog, 나머지는 게시판 GELF

진행 방식

1. 두 종류 신호만 → 이벤트 없음(S8까지만 발동할 수 있음).
2. 15분 안에 경계 신호까지 세 종류 → S10 이벤트, 앱·경계 차단, 티켓.
3. 원복: 경계 차단 해제 → 앱 차단 해제 → 티켓 종료.

확인할 결과물

- 세 계층의 신호가 같은 src_ip로 정규화됐는지(pfSense 로그의 IP 필드 이름 확인)
- pfSense 규칙 추가·해제 기록
- S8과 S10이 같은 침해를 각각 티켓으로 만들지 않는지

먼저 정할 것 — 규칙 개수가 3개라도 실제로 서로 다른 계층에서 왔는지 확인해야 한다. NAT·프록시 때문에 계층마다 보이는 출발지가 다를 수 있다.

통합 스토리라인과 최종 결과물

마지막 단계에서는 케이스를 따로 돌리지 않고, 한 번의 침해 흐름으로 이어서 재생해 인시던트 티켓 1건으로 모이는지 확인한다.

통합 침해 스토리라인: 한 번의 공격이 4계층을 관통한다

각 단계가 어느 계층에서 어떤 케이스로 잡히는지 · 마지막에는 여러 케이스가 한 인시던트 티켓으로 모인다

단계	공격 행위	계층	신호	탐지	대응
1	**엣지 스캔·악성 요청**	엣지	`cf-waf`	엣지 WAF 다중 룰	**113 통지**
2	**포트·웹 스캔**	네트워크	`pfsense-block · web-scan`	S10 card(rule)	**112+114+pf**
3	**저속·연쇄 브루트포스**	앱	`login-bruteforce`	S1 · S8	**111 / 112+114**
4	**계정 탈취(실패 뒤 성공)**	앱	`login-success`	S4 card(outcome)	**111+114**
5	**권한 상승**	앱	`priv-unauthorized-admin`	S9	**108 회수**
6	**앱 우회 민감 조회**	데이터	`MySQL 감사 (룰 100250)`	데이터 계층 룰	**114 티켓**
7	**전 계층 동시 지표**	네트워크	①~④ 다계층	확장 card(rule)	**112+114+pfSense**

결과: 엣지 샘플 → 스캔 → 브루트포스 → 계정 탈취 → 권한 상승 → DB 조회가 인시던트 티켓 1건으로 묶여야 한다

롤백 순서: IP 차단 해제 → 계정 unlock → 권한 회수·티켓 종료 → 엣지 샘플·감사 로그·룰 원복

최종 제출물 체크리스트

결과물	내용
케이스별 검증 기록 10장	3장의 기록 틀, 정상·임계 미만·충족·혼합 결과
Graylog 이벤트 정의	S1~S10 조건·창·묶기 설정 내보내기
n8n 워크플로	108·111·112·113·114 대응과 해제 흐름 내보내기
Wazuh 규칙·디코더	FIM·다단계·데이터 계층 규칙 파일
증적 패키지	실행 시각, 입력 조건, 로그, 이벤트, 티켓, 원복 결과를 하나로 연결
플레이북	케이스별 대응·원복 절차, 사람이 검토할 경계
재현 문서	다른 팀원이 따라 해서 같은 결과를 얻을 수 있는 절차

완료 기준

- **수집**: 테스트 이벤트를 필수 필드와 함께 검색할 수 있다.
- **탐지**: 정상·임계 미만·충족·서로 다른 사용자/IP를 구분한다.
- **대응**: 대상 상태 변화와 티켓 생성을 확인한다.
- **원복**: 차단·잠금·권한이 사전 상태로 돌아온다.
- **재현**: 다른 팀원이 문서만 보고 같은 결과를 낸다.

팀이 먼저 정해야 할 것 · 출처

항목	관련 케이스	결정할 내용
로그인 신호의 `outcome` 필드	S4	필드 추가 또는 조건을 `card(rule)`로 변경
순서 확인	S4 · S8 · S9	"A 뒤에 B"를 판정할 방법(시각 비교·상관 ID)
`night_login` 생성 위치·시간대	S2	Graylog 파이프라인 또는 Wazuh, KST 기준 시간
출발지 IP 해석	S3 · S5 · S6 · S8 · S10	프록시 헤더 신뢰 여부, 테스트 출발지 구분 방법
`signal_kind` 경로와 묶기 키	S7	FIM·웹 접근을 같은 자산으로 잇는 기준
pfSense 로그 정규화	S10	filterlog의 IP 필드를 `src_ip`로 맞추기
중복 대응 방지	전체	한 침해를 한 티켓으로 묶는 키와 시간 범위
자동 대응 vs 사람 검토	전체	어떤 대응을 자동으로, 어떤 것을 승인 후 실행할지

출처

- 노션 [168] 파이널 프로젝트 캡스톤 발표자료(이미지 정리판) — 케이스 10종 표, 4계층·8종 기술, 신호 지도, 통합 스토리라인, 롤백·안전 원칙
- 노션 06 P3 캡스톤 공방전(40차시·80h) — 차시 구성(설계·구현·공방전 라운드·사후분석·포트폴리오)
- `C:\gov\aleph-soar-capstone\docs\` — 01 기획, 02 아키텍처, 03 시나리오(검증 기록 틀)
- `C:\gov\Git-Connection\_7_board_test\` — auth_controller.py, app.py, security_controller.py, gold_controller.py, admin_controller.py (신호 생성 위치)
- 케이스 원본 [153]~[162] 본문은 이번에 열람하지 못함 — 세부 절차는 원본과 대조해 보완